

Methoden benutzen

Instanzvariablen – Teil 2

Kapselung, Initialisierung und Vergleich

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

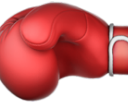
Prof. Dr. Martin Burger { 20. November 2025 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Methoden benutzen Instanzvariablen

Plan für diese Woche

 Objekte wissen und tun

 Methodenparameter

 Rückgabewerte

 Pass-by-Value

 Pair Programming

 Konventionen

 Getter und Setter

 Kapselung

 Variablen initialisieren

 Variablen vergleichen

 Pair Programming

 Spaß

📖 Methoden benutzen

🔗 Instanzvariablen

Verbindungen herstellen

- 👤 Tauschen Sie sich mit einer Nachbar:in aus!
- 🧑 Welche Konventionen kennen Sie schon, die eine professionelle, saubere Programmierung kennzeichnen?
- 🔄 Wozu werden Getter- und Setter-Methoden verwendet?
- 🤖 Wozu dient Kapselung?
- 👶 Was ist der Unterschied zwischen der Initialisierung von Instanzvariablen und lokalen Variablen?
- 👤 Was ist beim Vergleich von Variablen zu beachten?
- 🧑 Welche Fragen haben Sie zu diesem Kapitel?

4 Methoden benutzen Instanzvariablen

Wie sich Objekte verhalten



Zustand beeinflusst Verhalten, Verhalten beeinflusst Zustände. Wir wissen, dass Objekte über **Zustand** und **Verhalten** verfügen, die durch **Instanzvariablen** und **Methoden** repräsentiert werden. Dabei haben wir uns noch nicht angesehen, welche *Beziehung* Zustand und Verhalten zueinander haben. Wir wissen bereits, dass jede Instanz einer Klasse (jedes Objekt eines bestimmten Typs) seine eigenen, einmaligen Werte für Instanzvariablen besitzen kann. Hund (Dog-Objekt) A kann einen *Namen* haben, »Fido«, und ein *Gewicht* von 35 Kilo. Hund B heißt »Killer« und wiegt 5 Kilo. Wenn nun die Dog-Klasse eine Methode namens `makeNoise()` (`gibLaut()`) besitzt, sollte das Bellen eines 35-kg-Hunds dann nicht etwas tiefer sein als das seines deutlich kleineren Artgenossen? (Wobei wir davon ausgehen, dass ein nerviges Kläffen tatsächlich als *Bellen* durchgeht.) Glücklicherweise ist genau das der springende Punkt eines Objekts – es besitzt *Verhalten*, das sich auf seinen *Zustand* auswirkt. Anders gesagt, **Methoden benutzen die Werte von Instanzvariablen**. Ein Beispiel: »Wenn der Hund weniger als 7 Kilo wiegt, erzeuge ein Kläffgeräusch, sonst ...« oder »erhöhe das Gewicht um 5«. **Also los, lassen Sie uns ein paar Zustände ändern.**



Konventionen



Konventionen: Halten Sie sich an etablierte Standards zum Wohle aller!

„Don't get any ideas,“ the master tells her apprentice.



SHU

FOLLOW THE RULES



„Lehrling“
(Apprentice)

Der Lehrling befolgt jede Anweisung des Meisters, auch wenn sie zunächst unwichtig oder gar sinnlos erscheint. Der Meister wiederholt die Regeln so lange, bis der Lehrling sie verinnerlicht hat. Das Einhalten der Regeln ist wichtig für den Lernprozess.

Ilija Popjanev zu Shu Ha Ri  – frei übersetzt

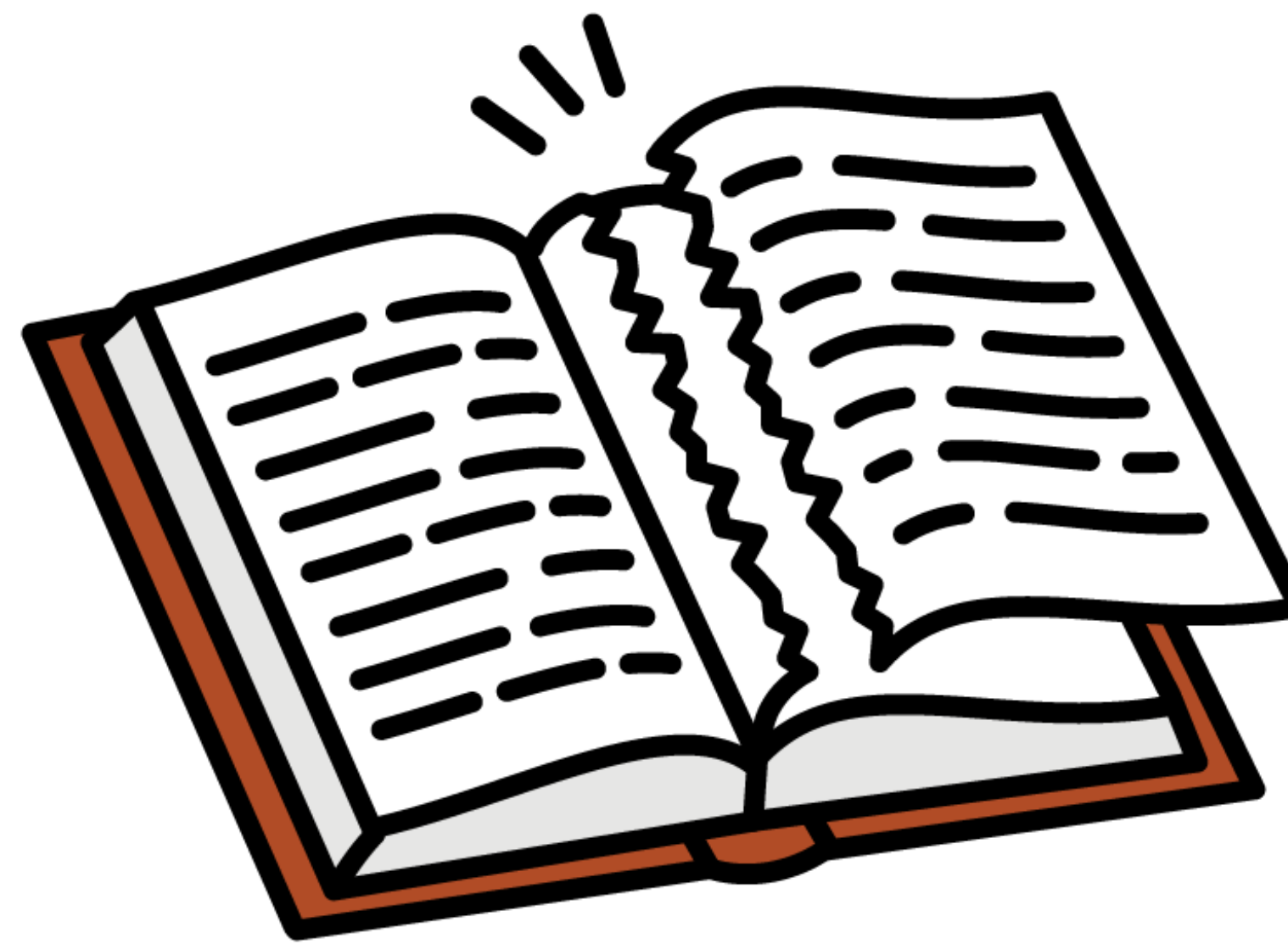


SHU

FOLLOW THE RULES



„Lehrling“
(Apprentice)

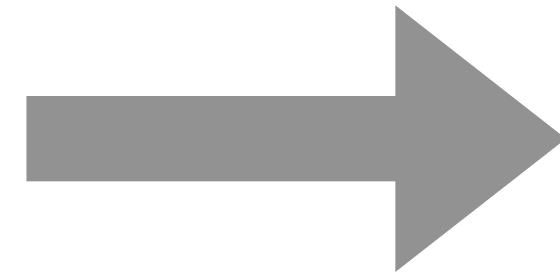


HA

BREAK THE RULES



„Geselle“
(Journeyman)



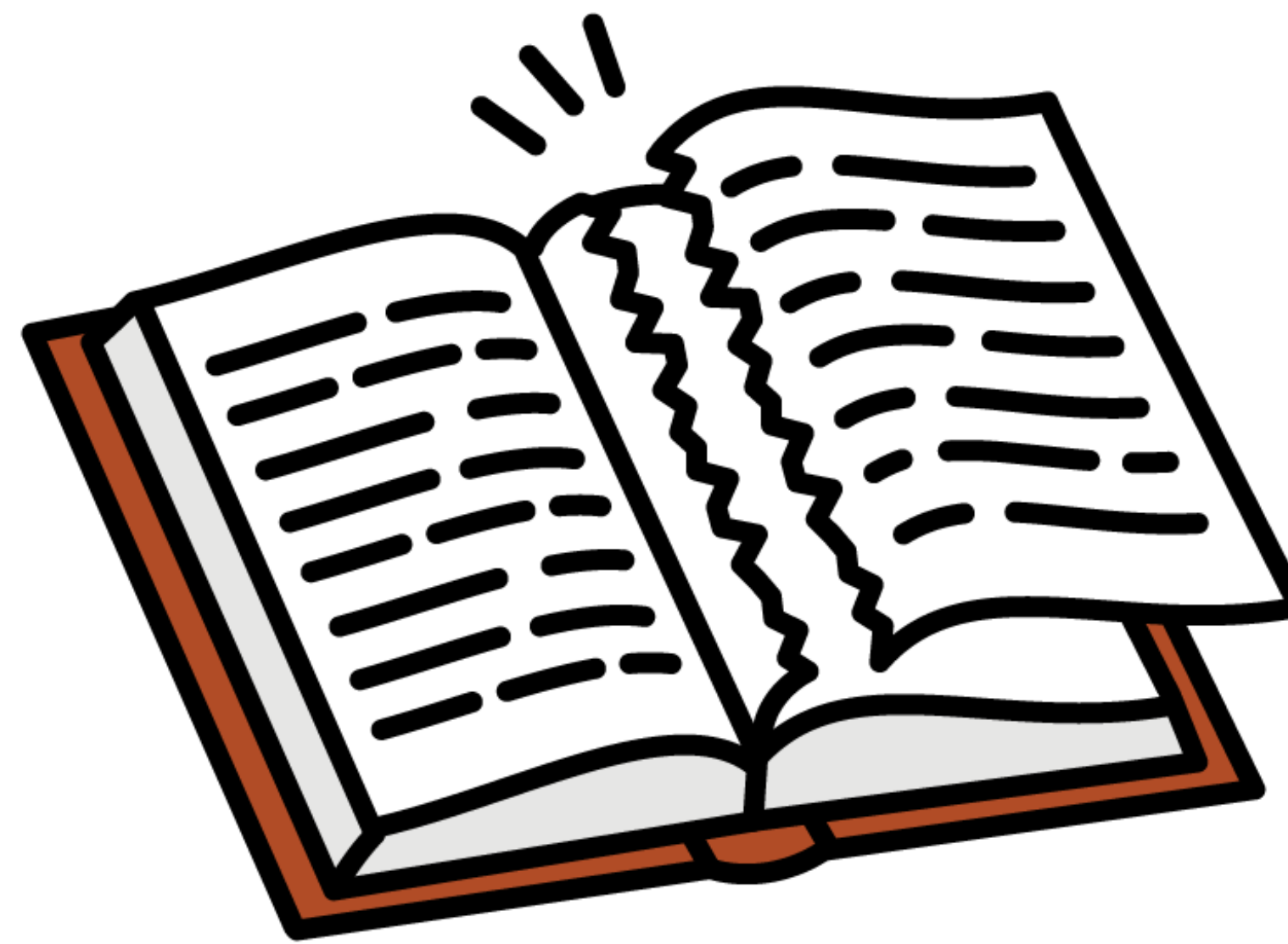
Der Geselle kann jetzt Regeln brechen. Er kann lehren, lernen, Feedback erhalten, seine Theorien erweitern und verbessern. Er kann Hypothesen aufstellen, experimentieren und neue Prinzipien einführen – **er muss sie nun selbst in der Praxis erfahren**, ohne die Hilfe seines Meisters.

Ilija Popjanev zu Shu Ha Ri  – frei übersetzt



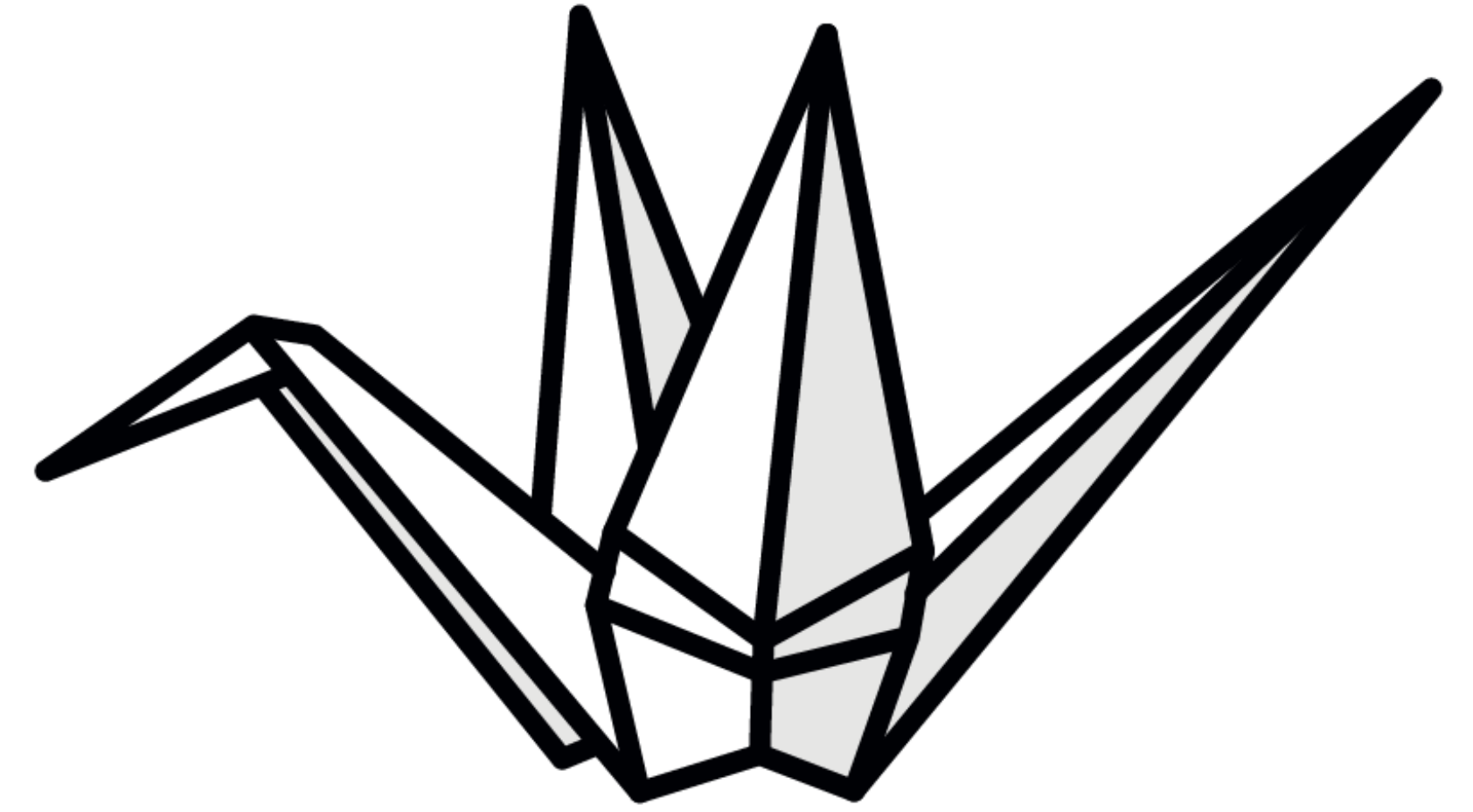
SHU

FOLLOW THE RULES



HA

BREAK THE RULES

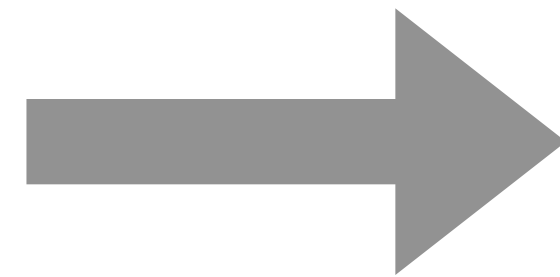


RI

TRANSCEND THE RULES



„Lehrling“
(Apprentice)



„Geselle“
(Journeyman)



„Meister“
(Master)

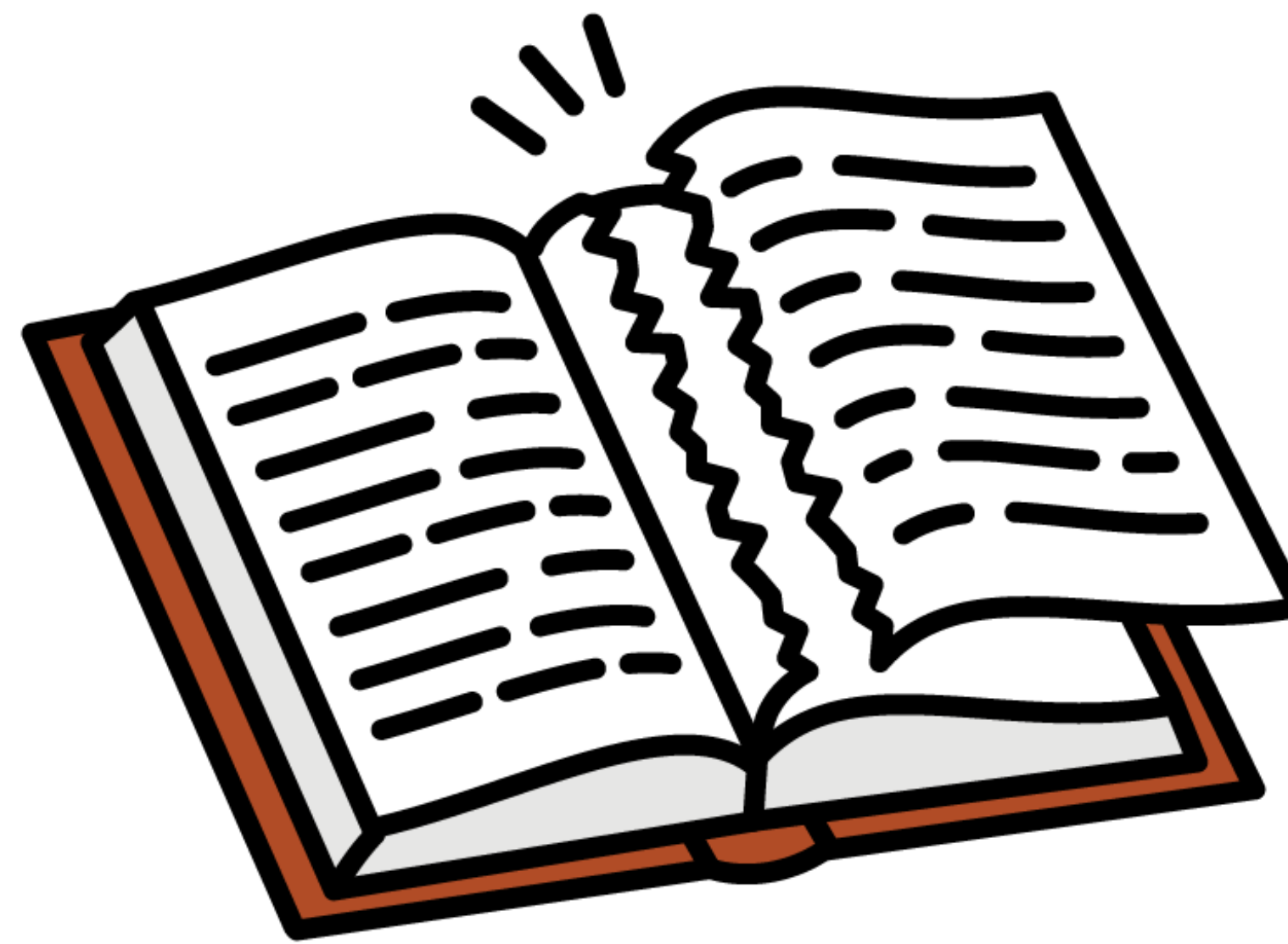
Der Meister ist ein Lernender, der experimentiert, sich verbessert, Regeln, Praktiken und Prinzipien entwickelt und Erfahrungen für seine Selbstentfaltung nutzt. **Er passt sich instinktiv seiner Umgebung an und bringt alle in einen Zustand hoher Leistung.**

Ilija Popjanev zu Shu Ha Ri  – frei übersetzt



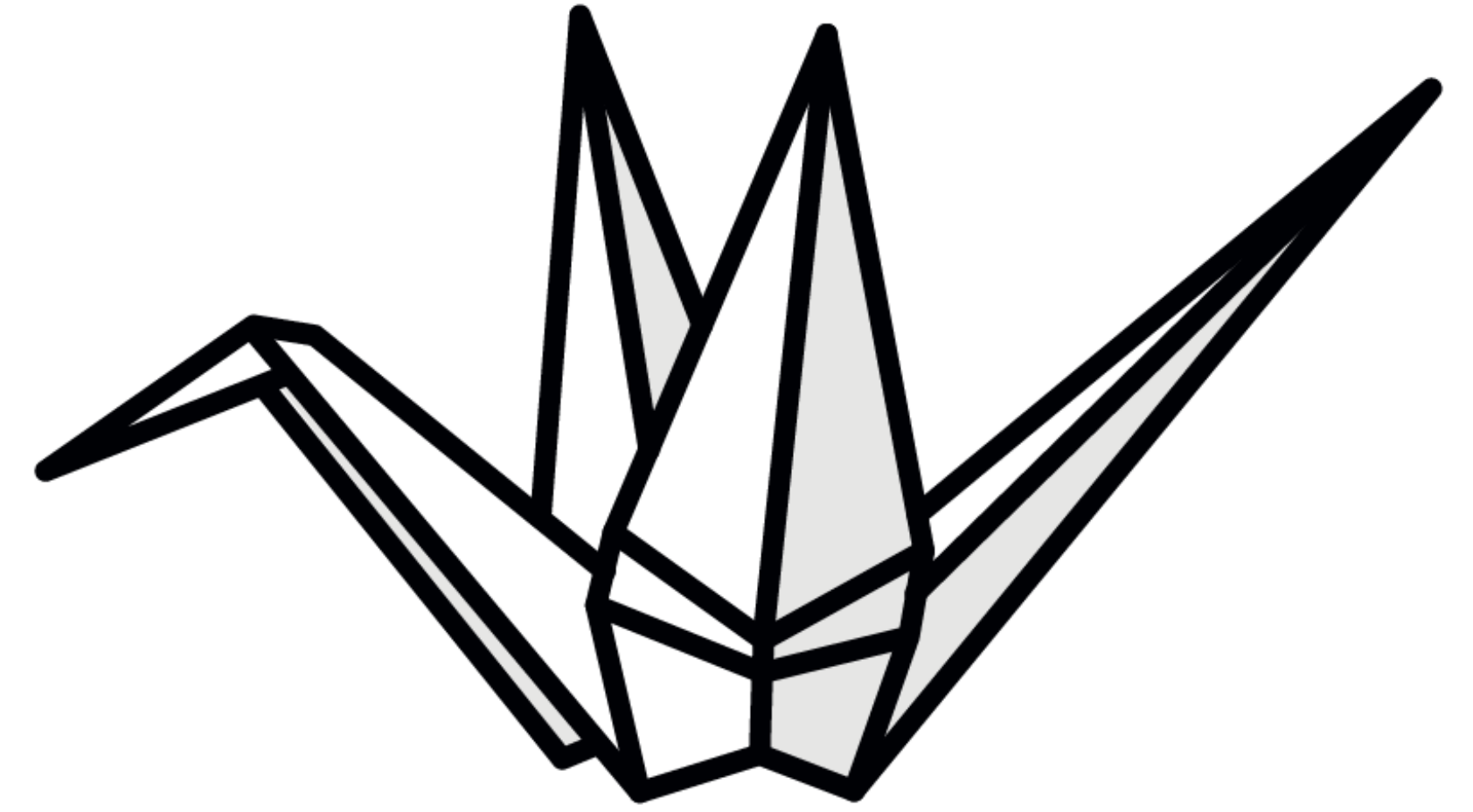
SHU

FOLLOW THE RULES



HA

BREAK THE RULES

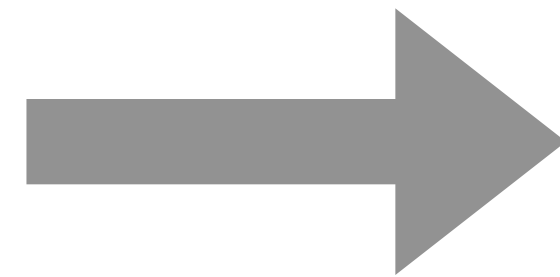


RI

TRANSCEND THE RULES



„Lehrling“
(Apprentice)



„Geselle“
(Journeyman)



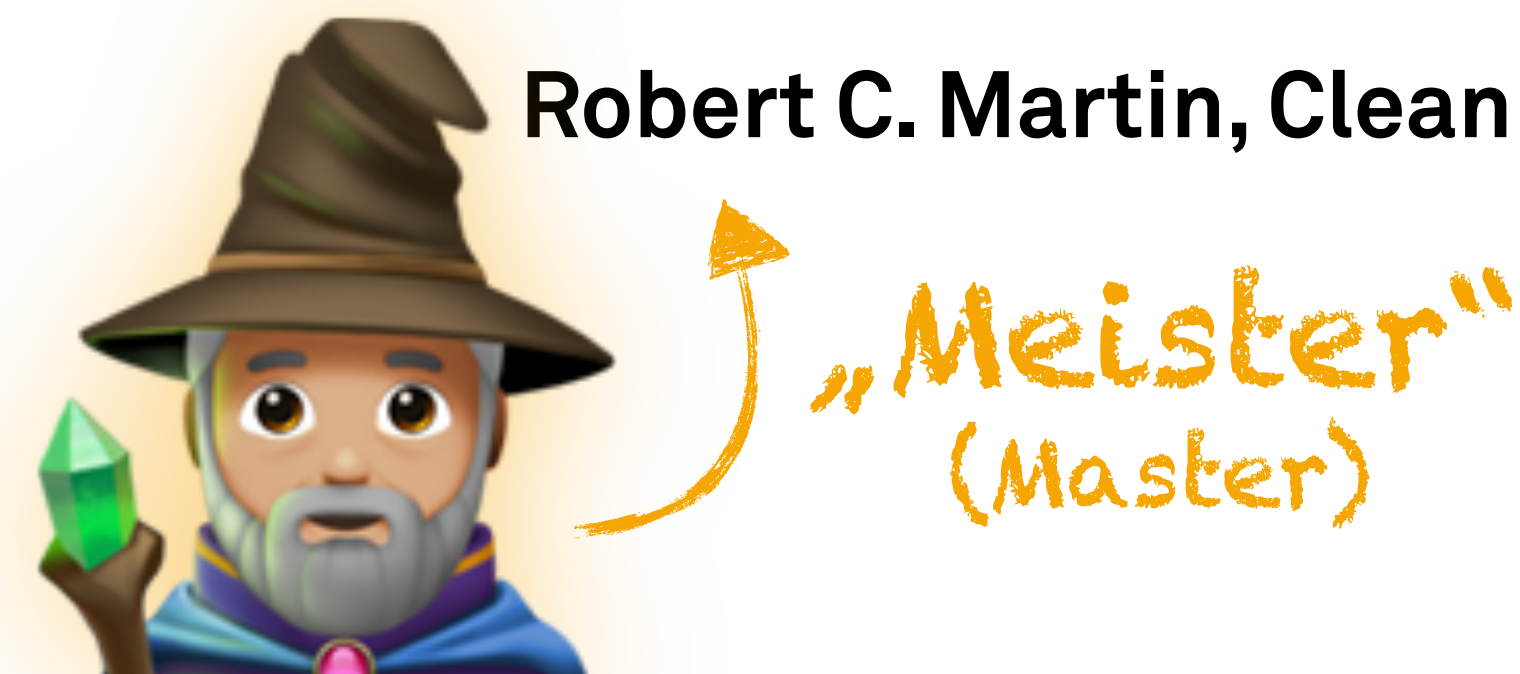
„Meister“
(Master)

Profi
(Beginn)

Profi
(Festigung)

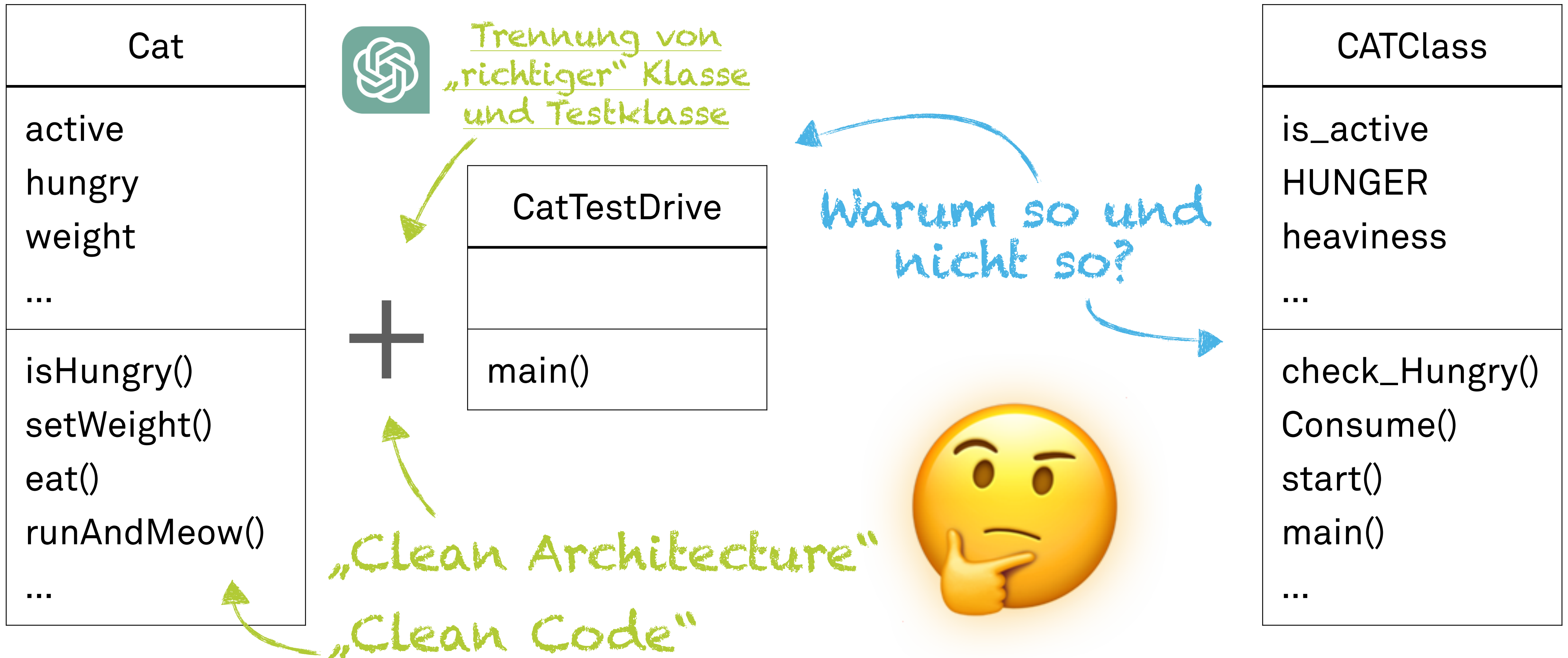
„Ein Unterschied zwischen einer smarten Programmierer:in und einer professionellen Programmierer:in ist, dass der **Profi** weiß, dass **Klarheit König** ist. **Profis** setzen ihre Fähigkeiten für das Gute ein und schreiben Code, den andere verstehen können.“

Robert C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship – frei übersetzt



Dinge benennen und strukturieren

Konventionen



„Ich habe gehört, dass Programmierer:innen viel mehr damit beschäftigt sind, anderen Code zu lesen und zu verstehen, als selbst neuen Code zu schreiben. Deshalb soll ich sauberen Code schreiben und mich an allgemein anerkannte Konventionen halten.“

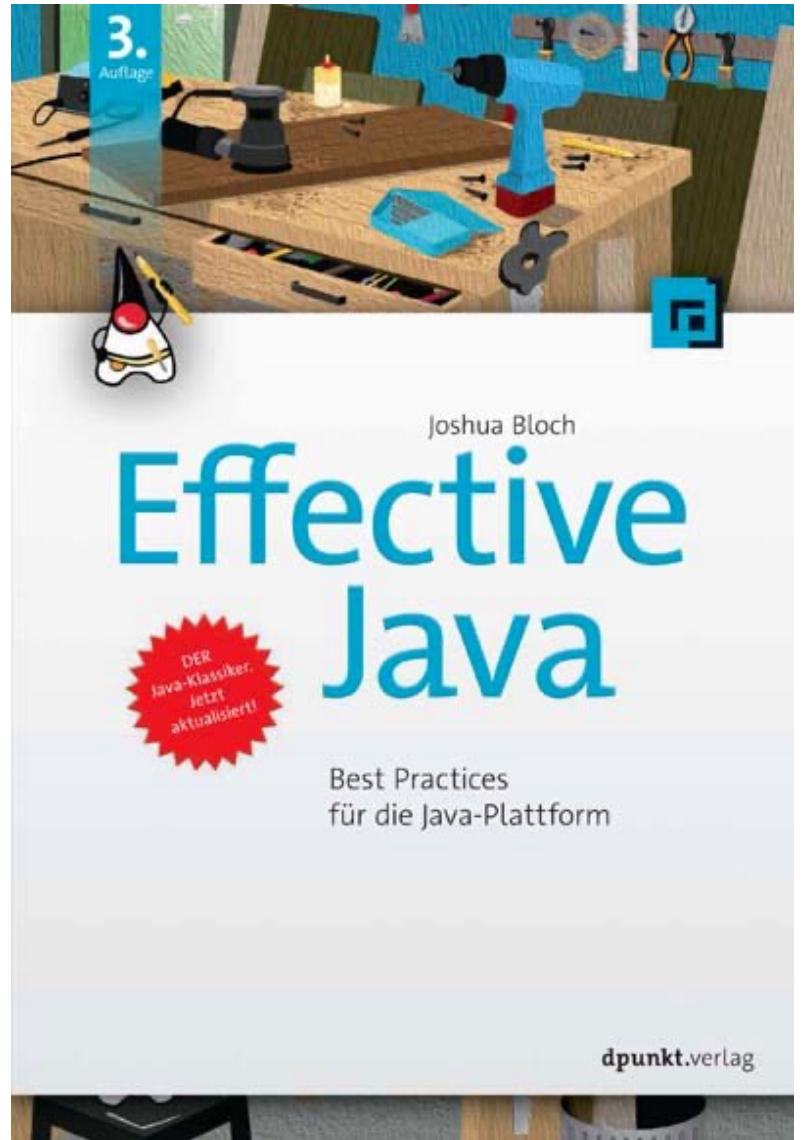
Noel – neugierige Student:in im 1. Semester

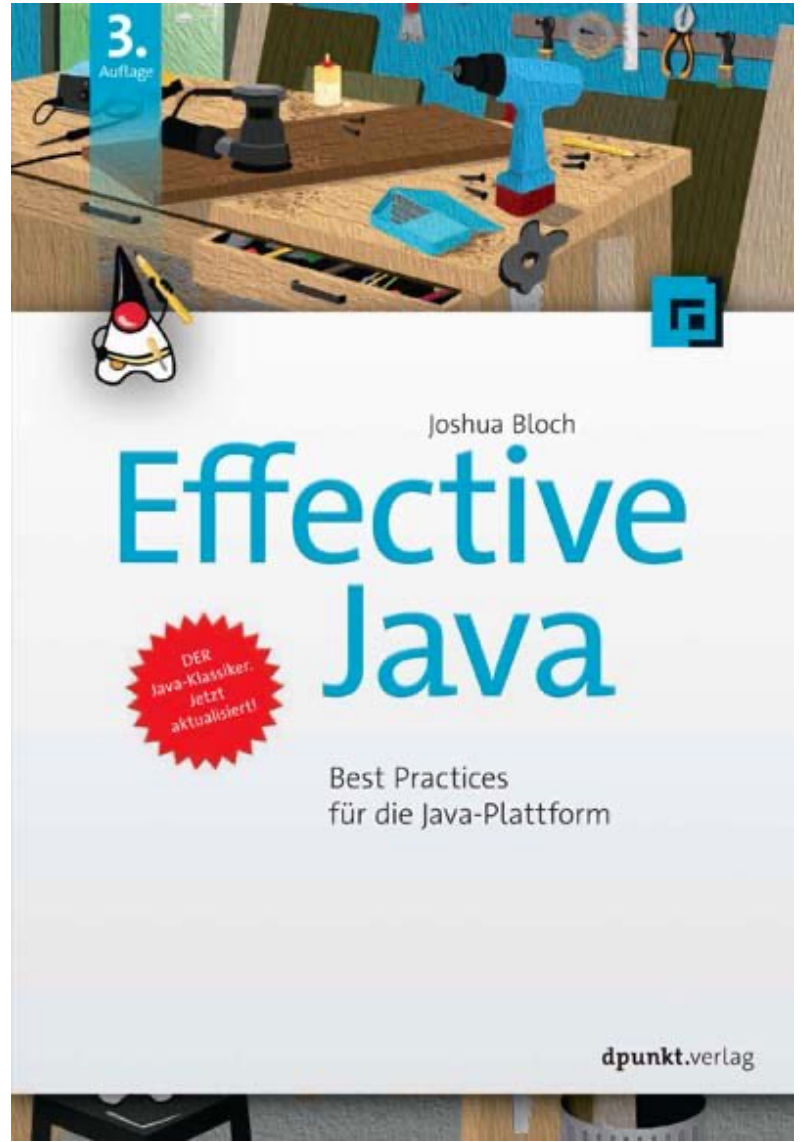


„Das stimmt absolut. Sauberer und lesbarer Code spart langfristig Zeit und Aufwand. Die Einhaltung bewährter Konventionen hilft, Code zu schreiben, den andere Entwickler:innen schnell verstehen können.“

Maxi – professionelle Entwickler:in, die andere zu Spitzenleistungen führt







! Quiz

🧙 Konventionen

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Programmierer:innen schreiben viel mehr neuen Code, als dass sie bestehenden Code analysieren und verstehen.

Nein, es ist genau umgekehrt.

Profis wissen, dass Klarheit König ist.
Deshalb schreiben sie Code, den andere
verstehen können.

Richtig!

Die Trennung von Implementierungscode (z.B. `Calculator`) und Testcode (z.B. `CalculatorTestDrive`) ist eine bewährte Praxis, die den Entwicklungsprozess und die Qualität des Codes in der Softwareentwicklung erheblich verbessert.

Richtig!

Namenskonventionen für Klassen, Methoden und Variablen  machen den Code lesbarer und verständlicher. Sie helfen z.B. auf einen Blick zu erkennen, ob es sich um eine Klasse, eine Methode oder eine Variable handelt.

Richtig!

 Welcher **eine** Gedanke zu
„ Konventionen“ war für Sie
am wichtigsten?  Schreiben
Sie ihn auf!

 Durchatmen und  Sortieren

Getter und Setter

Auslesen und Schreiben

Getter und Setter

- **Getter** und **Setter** sind...
 - ... eine Möglichkeit der Anwendung von **Rückgabetypen** und **Parametern**.
 - ... Methoden für das **Auslesen** (get) und **Setzen** (set) von Instanzvariablen.
- Getter: Gibt den Wert einer Instanzvariablen als den **Rückgabewert** zurück.
- Setter: Verwendet den **Argumentwert**, um den Wert einer Instanzvariablen zu setzen.

don't
just
take,
give

Auslesen und Schreiben

Getter und Setter

```
class Song {  
    String artist;  
    String title;  
  
    String getArtist() {  
        return artist;  
    }  
  
    void setArtist(String newArtist) {  
        artist = newArtist;  
    }  
    // ...  
}
```

Instanzvariable

Getter
und
Setter

Song
artist title
getArtist() setArtist() getTitle() setTitle() play()

! ? Quiz

🔄 Getter und Setter

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Getter-Methoden werden verwendet, um den Wert eines Attributs zu lesen.

Richtig!

Getter-Methoden beginnen in der Regel mit `get`, gefolgt vom Namen des Attributs. Zum Beispiel `getAge()`.

Richtig!

Setter-Methoden beginnen in der Regel mit `mod`, gefolgt vom Namen des Attributs. Zum Beispiel `modAge(int newAge)`.

Sie beginnen normalerweise mit `set`.

 Welcher **eine** Gedanke zu
„ Getter und Setter“ war für
Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

„Okay... Und **wozu** der ganze Aufwand?“

Noel – neugierige Student:in im 1. Semester

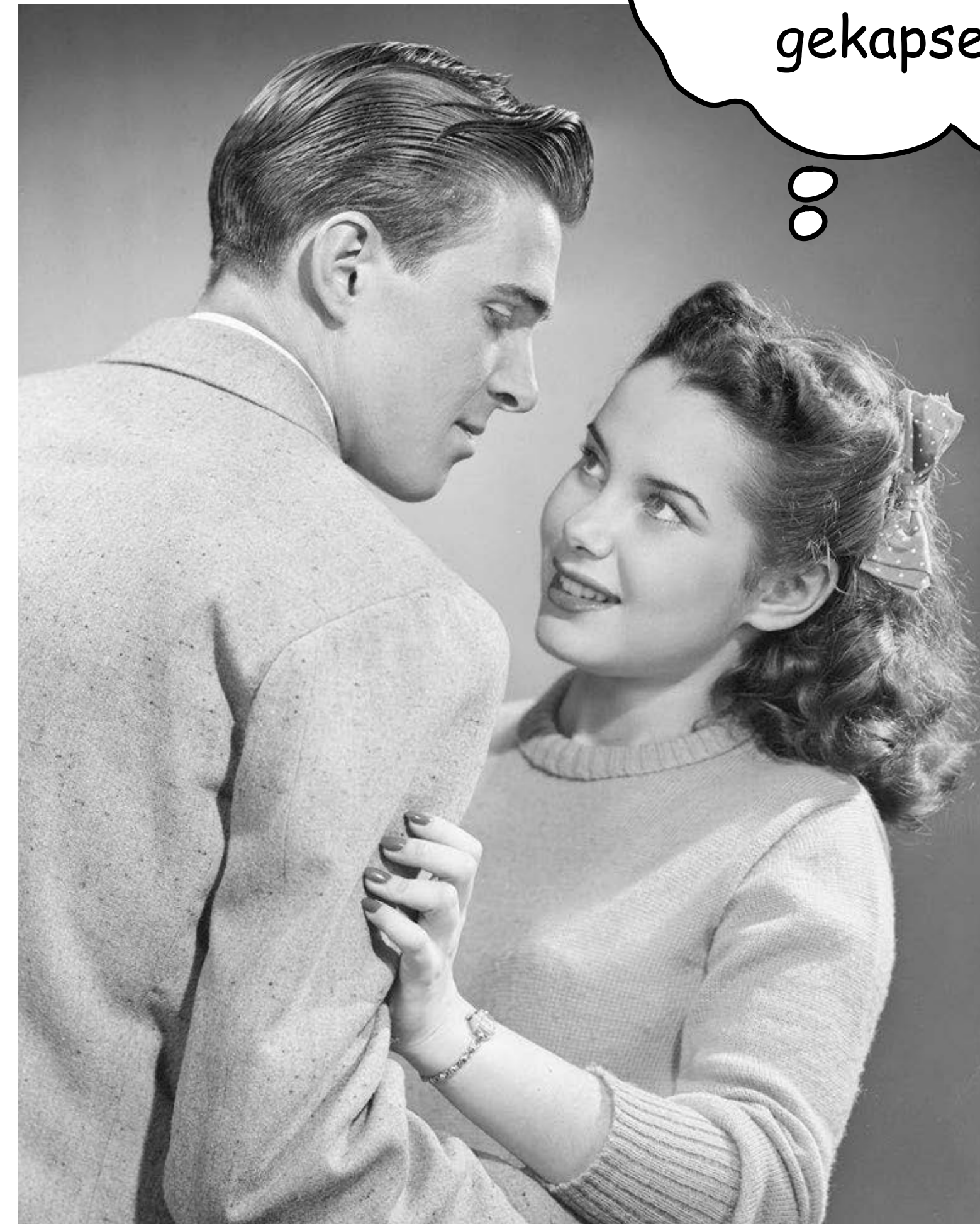


Kapselung

Doh! So nicht...

🤖 Kapselung

```
class TunedCar {  
  
    int fuelLevel; // %  
    int topSpeed; // mph  
  
    // ...  
}  
  
car.fuelLevel = 101;  
car.topSpeed = 200;
```



Jenny sagt, dass
du ordentlich
gekapselt bist ...

Uh-uh!... fast...

👁️ Kapselung

```
class StreetLegalCar {  
  
    int fuelLevel; // %  
    int topSpeed; // mph  
  
    void setFuelLevel(int level) {  
        if (level ≥ 0 && level ≤ 100) {  
            fuelLevel = level;  
        }  
    }  
  
    void setTopSpeed(int speed) {  
        // highest posted speed limit in the United States  
        if (speed ≥ 0 && speed ≤ 85) {  
            topSpeed = speed;  
        }  
    }  
}
```



Nicht so schnell!

Jenny

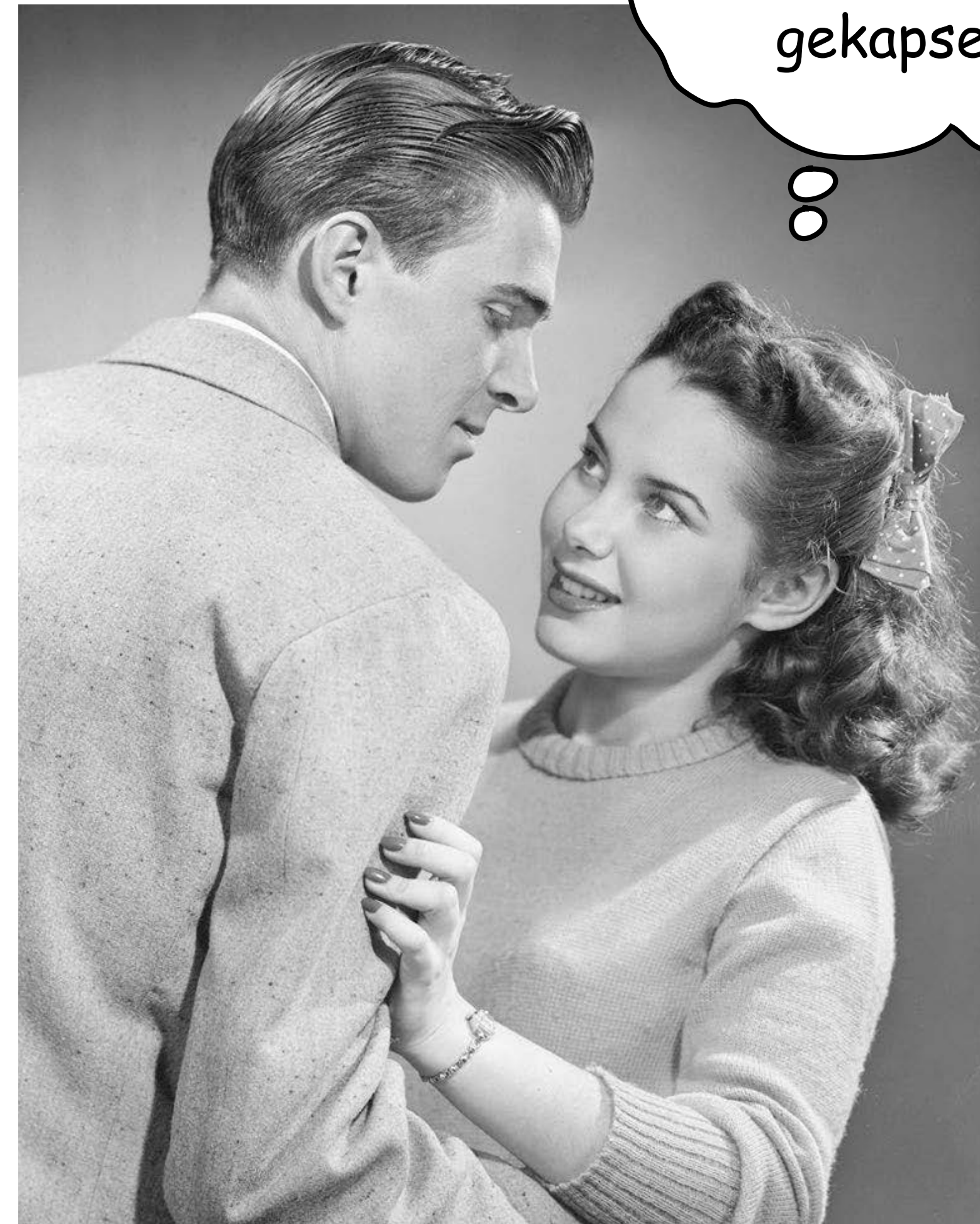
Mit Settern können wir Tests einbauen, die die Sicherheit des Autos gewährleisten.

Whew! ... aber so!

🤖 Kapselung

So erzwingen wir,
dass die Setter
aufgerufen
werden.

```
class StreetLegalCar {  
    private int fuelLevel; // %  
    private int topSpeed; // mph  
  
    public void setFuelLevel(int level) {  
        if (level ≥ 0 && level ≤ 100) {  
            fuelLevel = level;  
        }  
    }  
  
    public void setTopSpeed(int speed) {  
        // highest posted speed limit in the United States  
        if (speed ≥ 0 && speed ≤ 85) {  
            topSpeed = speed;  
        }  
    }  
}
```



Jenny sagt, dass
du ordentlich
gekapselt bist ...

```
1 class TunedCar {
2
3     int topSpeed; // max mph
4
5     int speed; // current mph
6
7     void accelerate(int acceleration) {
8         System.out.println("INFO - Acceleration by: " + acceleration);
9         int newSpeed = speed + acceleration;
10        if (newSpeed <= topSpeed) {
11            speed = newSpeed;
12            System.out.println("INFO - ... new speed: " + newSpeed);
13        } else {
14            System.out.println("WARN - ... ignoring acceleration to: " + newSpeed);
15            System.out.println("WARN - ... would be more than: " + topSpeed);
16        }
17    }
18 }
19
```

STDIN

Input for the program (Optional)

Output:

```
Creating a new tuned car...
Setting the top speed to 80...
Accelerate by 70...
INFO - Acceleration by: 70
INFO - ... new speed: 70
Accelerate by 130...
INFO - Acceleration by: 130
WARN - ... ignoring acceleration to: 200
WARN - ... would be more than: 80
Setting the top speed to 200...
Accelerate by 130...
INFO - Acceleration by: 130
INFO - ... new speed: 200
```



```
1 class SeeminglyStreetLegalCar {
2
3     int topSpeed; // max mph
4
5     int speed; // current mph
6
7     int getTopSpeed() {
8         return topSpeed;
9     }
10
11    void setTopSpeed(int newTopSpeed) {
12        // highest posted speed limit in the United States
13        if (newTopSpeed >= 0 && newTopSpeed <= 85) {
14            topSpeed = newTopSpeed;
15            System.out.println("INFO - New top speed: " + newTopSpeed);
16        } else {
17            System.out.println("WARN - Ignoring new top speed: " + newTopSpeed);
18        }
19    }
20
21    void accelerate(int acceleration) {
22        System.out.println("INFO - Acceleration by: " + acceleration);
23        int newSpeed = speed + acceleration;
24        if (newSpeed <= topSpeed) {
25            speed = newSpeed;
26            System.out.println("INFO - ... new speed: " + newSpeed);
27        } else {
28            System.out.println("WARN - ... ignoring acceleration to: " + newSpeed);
29            System.out.println("WARN - ... would be more than: " + topSpeed);
30        }
31    }
32 }
33
```

STDIN

Input for the program (Optional)

Output:

```
Creating a seemingly street legal car...
Setting the top speed to 80...
Accelerate by 70...
INFO - Acceleration by: 70
INFO - ... new speed: 70
Accelerate by 130...
INFO - Acceleration by: 130
WARN - ... ignoring acceleration to: 200
WARN - ... would be more than: 80
Setting the top speed to 200...
Accelerate by 130...
INFO - Acceleration by: 130
INFO - ... new speed: 200
```



```
1 class StreetLegalCar {
2
3     private int topSpeed; // max mph
4
5     private int speed; // current mph
6
7     public int getTopSpeed() {
8         return topSpeed;
9     }
10
11    public void setTopSpeed(int newTopSpeed) {
12        // highest posted speed limit in the United States
13        if (newTopSpeed >= 0 && newTopSpeed <= 85) {
14            topSpeed = newTopSpeed;
15            System.out.println("INFO - New top speed: " + newTopSpeed);
16        } else {
17            System.out.println("WARN - Ignoring new top speed: " + newTopSpeed);
18        }
19    }
20
21    public void accelerate(int acceleration) {
22        System.out.println("INFO - Acceleration by: " + acceleration);
23        int newSpeed = speed + acceleration;
24        if (newSpeed <= topSpeed) {
25            speed = newSpeed;
26            System.out.println("INFO - ... new speed: " + newSpeed);
27        } else {
28            System.out.println("WARN - ... ignoring acceleration to: " + newSpeed);
29            System.out.println("WARN - ... would be more than: " + topSpeed);
30        }
31    }
32 }
33
```

STDIN

Input for the program (Optional)

Output:

```
Creating a street legal car...
Setting the top speed to 80...
INFO - New top speed: 80
Accelerate by 70...
INFO - Acceleration by: 70
INFO - ... new speed: 70
Accelerate by 130...
INFO - Acceleration by: 130
WARN - ... ignoring acceleration to: 200
WARN - ... would be more than: 80
Setting the top speed to 200...
WARN - Ignoring new top speed: 120
Accelerate by 130...
INFO - Acceleration by: 130
WARN - ... ignoring acceleration to: 200
WARN - ... would be more than: 80
```

Markieren Sie **Instanzvariablen** als **private**!
Markieren Sie **Getter** und **Setter** als **public**!

Keep out of harm's way!

! Quiz

🧠 Kapselung

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Mit Hilfe von Kapselung können Sie steuern, wer die Daten in Ihrer Klasse auf welche Weise ändern kann.

Richtig!

Setter-Methoden dürfen keinen Validierungscode enthalten, der sicherstellt, dass die Instanzvariablen nicht in unzulässiger Weise verändert werden.

Dies ist einer der Hauptvorteile von Settern gegenüber direkten Attributzugriffen.

Ein Vorteil der konsequenten Einführung von Gettern und Settern ist, dass Sie Ihre Meinung später ändern können, ohne viel Code anpassen zu müssen.

Richtig!

Markieren Sie eine Instanzvariable als `public`, damit sie nicht durch direkten Zugriff geändert werden kann.

Markieren Sie zu diesem Zweck eine Instanzvariable als `private`.

Der Zugriff auf private Attribute ist nur innerhalb derselben Klasse möglich. Dies ist ein fundamentales Prinzip der Datenkapselung in der objektorientierten Programmierung.

Richtig!

Getter- und Setter-Methoden sollten immer den Zugriffsmodifikator `private` haben.

Getter und Setter sind typischerweise `public`, so dass andere Klassen darauf zugreifen können.

 Welcher **eine** Gedanke zu
„ Kapselung“ war für Sie am
wichtigsten?  Schreiben Sie
ihn auf!

 Durchatmen und  Sortieren

„Mmh... Und was passiert, wenn ich eine Getter-Methode auf einer nicht initialisierten Instanzvariablen aufrufe?“

Merle – aufmerksame Student:in im 1. Semester



Variablen initialisieren

Variablendeklaration

👶 Variablen initialisieren

- Zur Erinnerung:
 - Für eine Variablendeklaration brauchen Sie zumindest Name und Typ:
- String name;
- Sie können die Variable gleichzeitig initialisieren, ihr also einen Wert zuweisen:

```
String name = "A. N. Other";
```



„Ja, ja, ich weiß! Welchen Wert hat denn nun eine **Instanzvariable** vor ihrer Initialisierung?“

Ulli – ungeduldige Student:in im 1. Semester



Instanzvariablen haben immer einen
Standardwert...


```
1 class InstanceVariables {
2
3     // Declarations without explicit initialization:
4     // - We define both the type and the name of each instance variable.
5     // - We do not assign an explicit value.
6     private int i;
7     private float f;
8     private char c;
9     private boolean b;
10    private String s;
11
12    // What might these getters return?
13
14    public int getInt() {
15        return i;
16    }
17
18    public float getFloat() {
19        return f;
20    }
21
22    public char getChar() {
23        return c;
24    }
25
26    public boolean getBoolean() {
27        return b;
28    }
29
30    public String getString() {
31        return s;
32    }
33 }
34
```

STDIN

Output:

Click on RUN button to see the output

Instanzvariablen haben immer einen
Standardwert:
0 (einschließlich char), false oder null.

Instanzvariablen müssen nicht – im Gegensatz zu lokalen Variablen – zwingend initialisiert werden.

„Ha... Es gibt also verschiedene Arten von Variablen?“

Gerit – aufgeweckte Student:in im 1. Semester




```
class Horse {  
    private double height = 15.2;  
    private String breed;  
    // ...  
}
```

Instanzvariablen werden innerhalb einer Klasse, aber nicht innerhalb einer Methode deklariert.

```
class AddThing {  
    int a;  
    int b = 12;  
  
    public int add() {  
        int total = a + b;  
        return total;  
    }  
}
```

Lokale Variablen werden innerhalb einer Methode deklariert.



Foo und
Bar?

```
class Foo {  
    public void go() {  
        int x;  
        int z = x + 3;  
    }  
}
```



```
Foo.java:4: error: variable x  
might not have been initialized  
        int z = x + 3;  
                  ^  
1 error
```



Lokale Variablen müssen vor ihrer Verwendung initialisiert werden.

! ? Quiz

👶 Variablen initialisieren

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Lokale Variablen, beispielsweise innerhalb von Methoden, erhalten Standardwerte, auch wenn Sie nicht explizit einen Wert zuweisen.

Instanzvariablen werden auch ohne explizite Zuweisung eines Wertes mit Standardwerten belegt.

Die Initialisierung von Instanzvariablen bei der Deklaration ist optional.

Richtig!

Lokale Variablen erhalten automatisch den Wert `null`, wenn sie nicht initialisiert werden.

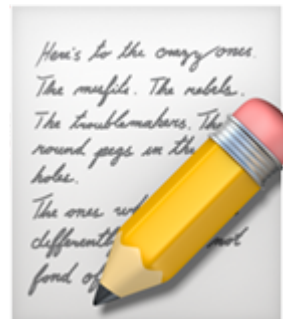
Lokale Variablen haben keinen Standardwert.

Lokale Variablen müssen immer explizit initialisiert werden.

Richtig!

Die Parameter einer Methode sind ebenfalls lokale Variablen. Sie müssen daher immer explizit von Ihnen initialisiert werden, da sonst ein Compilerfehler auftritt.

Da der Compiler sicherstellt, dass Methoden stets mit passenden Argumenten aufgerufen werden, sind Parameter immer initialisiert.

 Welcher **eine** Gedanke zu
„ Variablen initialisieren“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

Variablen vergleichen

=-Operator

👤 Variablen vergleichen

- Mit dem ==-Operator können Sie herausfinden, ob zwei **elementare Werte gleich** sind.
- Bei zwei Objekten überprüft dieser Operator, ob es die beiden **selben Objekte auf dem Heap** sind. Dann verweisen die Referenzen der beiden Objekte auf das selbe Objekt in der Speicherverwaltung.
- Ob zwei Objekte als gleich behandelt werden, hängt davon ab, was für diesen bestimmten Objekttyp Sinn ergibt.
 - Das Konzept der Objektgleichheit  behandeln wir später.



Für den Moment ist es wichtig zu verstehen, dass der Operator `=` nur für den Vergleich von Bits in zwei Variablen verwendet wird. Was diese Bits darstellen, spielt keine Rolle. Die Bits sind entweder gleich oder ungleich.

! ? Quiz

👤 Variablen vergleichen

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Verwenden Sie $=$, um zu testen, ob zwei elementare Variablen den gleichen Wert haben.

Richtig!

Verwenden Sie `=`, um zu testen, ob zwei Referenzvariablen übereinstimmen, also auf zwei Objekte mit gleichen Daten verweisen.

Dies prüft, ob zwei Referenzvariablen auf das selbe Objekt auf dem Heap zeigen.

Verwenden Sie `equals()`, um zu prüfen, ob zwei Objekte gleich sind (aber nicht unbedingt dasselbe Objekt), zum Beispiel um zu testen, ob zwei `String`-Objekte die gleiche Zeichenfolge enthalten.

Richtig!

 Welcher **eine** Gedanke zu
„ Variablen vergleichen“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

Pair Programming

🎓 **Kapselung und Initialisierung**

🤝 **Pair Programming**

🎯 **Implementieren Sie einen arithmetischen Dividierer als objektorientiertes Programm!**

✅ Zustand: Dividend und Divisor

✅ Verhalten: Teilt Dividend durch Divisor

✅ Die Methode `divide()` gibt das Ergebnis zurück (`dividend / divisor`).

✅ Division durch 0 soll verhindert werden.



Kapselung und Initialisierung

Pair Programming

Implementieren Sie einen arithmetischen Dividierer als objektorientiertes Programm!

- ✓ Zustand: Dividend und Divisor
- ✓ Verhalten: Teilt Dividend durch Divisor
 - ✓ Die Methode `divide()` gibt das Ergebnis zurück (`dividend / divisor`).
 - ✓ Division durch 0 soll verhindert werden.

- ✓ Verwenden Sie Setter, Getter und die passenden Zugriffsmodifizierer.
- 🚩 Am Ende sollten Sie eine Klasse `Divider`
 - 🚩 Attribute: `dividend` und `divisor`
 - 🚩 Methoden: Getter, Setter und `divide()`
- 🚩 ... und eine Testklasse `DividerTestDrive` haben.

```
1 public class Divider {
2
3     private double dividend;
4     private double divisor = 1.0; // prevent division by zero
5
6     public double getDividend() {
7         return dividend;
8     }
9
10    public void setDividend(double d) {
11        dividend = d;
12    }
13
14    public double getDivisor() {
15        return divisor;
16    }
17
18    public void setDivisor(double d) {
19        if (d != 0.0) { // prevent division by zero
20            divisor = d;
21        }
22    }
23
24    public double divide() {
25        return dividend / divisor;
26    }
27 }
28
```

STDIN

Input for the program (Optional)

Output:

Test with default values...

 Expected: 0.0 – Actual: 0.0 Expected: 1.0 – Actual: 1.0 Expected: 0.0 – Actual: 0.0

Test with custom values...

 Expected: 10.0 – Actual: 10.0 Expected: 5.0 – Actual: 5.0 Expected: 2.0 – Actual: 2.0

Test whether divisor of 0 is prevented...

 Expected: 5.0 – Actual: 5.0

 Welcher **eine** Gedanke zu
„ Kapselung und
Initialisierung“ war für Sie am
wichtigsten?  Schreiben Sie ihn
auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Methoden benutzen Instanzvariablen

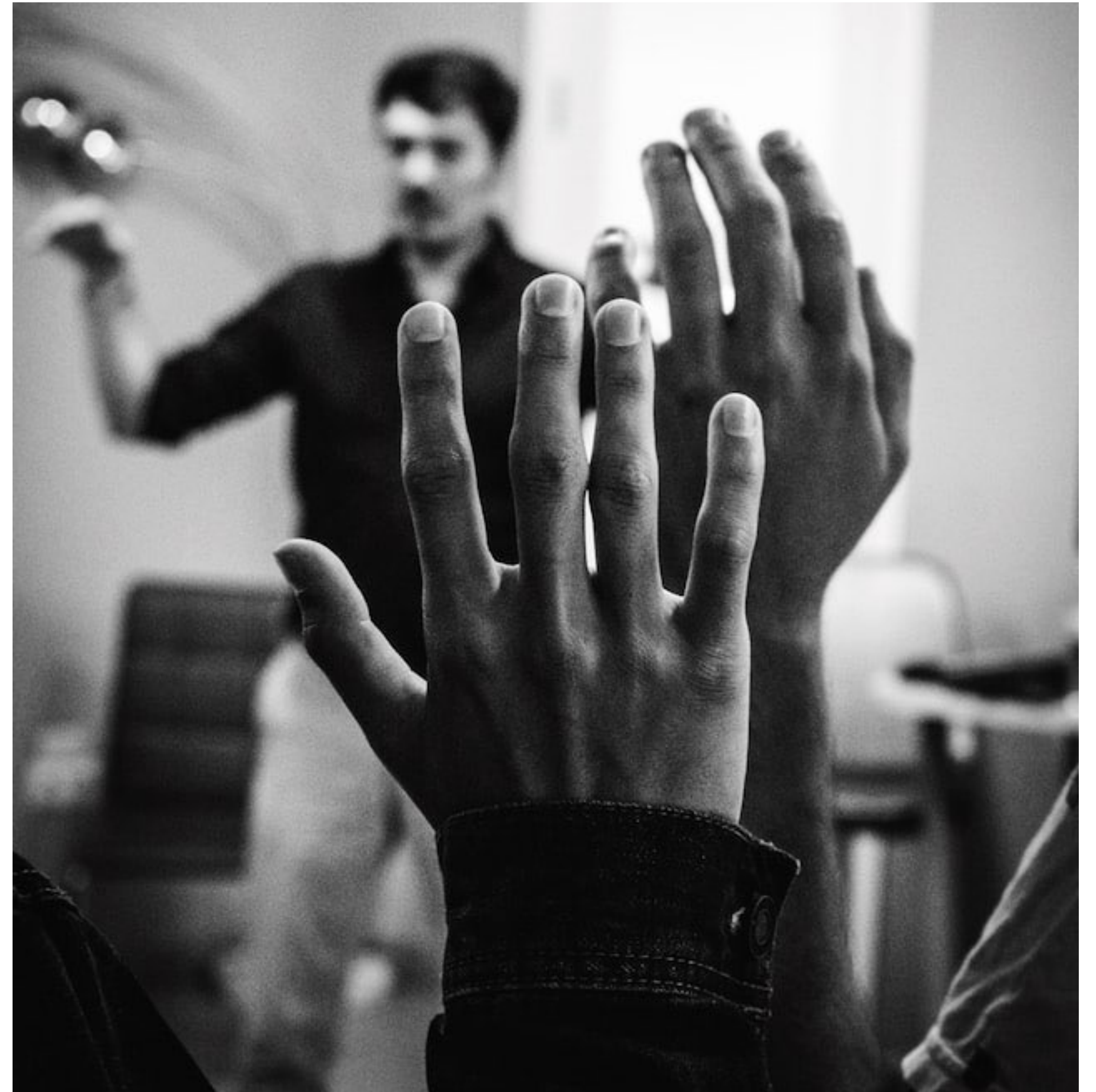
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Methoden benutzen Instanzvariablen?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Methoden benutzen Instanzvariablen



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

